

ORIGINAL ARTICLE

Efficient deep neural unification in symbolic processing

Hiroshi Honda¹  · Masafumi Hagiwara¹

Received: 22 April 2024 / Accepted: 9 July 2025 / Published online: 29 July 2025

© The Author(s) 2025

Abstract

Unification is indispensable for inferences in symbolic processing. The authors propose a neural network-based solution to perform efficient unification. Symbolic processing in conventional artificial intelligence has strong inference abilities but is not well suited for handling large amounts of ambiguous data. Contrastingly, neural networks can easily handle large amounts of ambiguous data but are not well suited for making complex inferences. Therefore, the authors realized the unification of the knowledge base, including ambiguous data, using a network that combines a memory network and recurrent neural network. The novelty of the proposed network is that matching, which is a unification process, is highly efficient and substitution, which is a unification process, is robust. The proposed network enables highly efficient matching by grouping multiple terms and processing them in a memory network. Furthermore, it can handle unknown words even during substitution because it uses a recurrent neural network to perform substitution. The experimental results show that the proposed network can achieve more efficient unification of ambiguous data than the baseline. This study combines symbolic processing and deep learning and suggests that it contributes to the realization of complex inferences from large amounts of ambiguous data, which has proven challenging in conventional research. Furthermore, the use of unification, which handles large amounts of ambiguous data, facilitates the development of inference systems with human-interactive interfaces. This allows humans to obtain inference results without knowing the representations of the knowledge base.

Keywords Neuro-symbolic AI · Deep learning · Unification · Prolog

1 Introduction

Symbolic processing in the field of artificial intelligence has been shown to have a strong inference ability. However, it cannot handle large amounts of ambiguous data because it requires the strict manual construction of knowledge bases. Research has also been conducted on symbolic processing to make inferences from incomplete data [1–5]; however, this is not well suited for handling large amounts of data on the web. By contrast, neural networks can easily learn from ambiguous data; however, they cannot make complex inferences. Therefore, studies have been conducted on symbolic processing using neural networks to make complex inferences from ambiguous data.

In the 1990s, research was conducted on symbolic processing using multilayer neural networks [6, 7]. However, practical results have not been achieved owing to the limitations of available hardware and the learning ability of multilayer neural networks.

In the 2000s, deep learning greatly improved the learning abilities of neural networks. Consequently, research on symbolic reasoning using neural networks [8] has greatly increased. Symbolic processing using deep learning is expected to yield complex inferences from massive amounts of ambiguous data.



In this study, unification, which is indispensable for inference using symbolic processing, was achieved using deep learning. Unification, which automatically determines whether terms match, is an integral process of inference using symbolic processing. Unification is the process of obtaining the same term by performing a substitution operation on both terms. For example, in logic programming, if there are two terms “*father(-tom,bob).*” and “*father(X,bob).*”, “*X = tom.*” can be derived through unification. The use of unification, which handles large amounts of ambiguous data, enables inference systems with human-interactive interfaces. In conventional symbolic processing, users are limited to a single query expression and cannot use a variety of phrases. This is inconvenient because users must remember how to express phrases in the knowledge base. To realize inference systems with user-friendly interactive interfaces, it is necessary to enable users to present queries using a variety of phrases through a unification that can handle ambiguous data.

In previous studies, matching [9], the process of determining whether unification is possible was conducted using deep learning. Previous studies have enabled matching even when unknown words are included in terms of deep learning. However, it is necessary to extract individual facts from a knowledge base and perform matching, which is computationally inefficient. This is because multiple terms cannot be grouped together and matching needs to be performed individually. In the recent era of big data, the size of the knowledge base to be handled has become extremely large; thus, efficient matching with the knowledge base is required. In addition, previous studies used deep learning only for matching and not for substitution, which is necessary for unification. If the substitution is rule-based, as in previous studies, the substitution of unknown words cannot be handled.

Therefore, we propose a network with features that have not been realized using conventional networks. The proposed network solves two problems of unification: improving the computational efficiency of matching and improving the robustness of substitution. The contributions of the proposed network are as follows:

1. Highly efficient matching by grouping multiple terms to efficiently handle large knowledge bases
2. Robust unknown word substitution by deep learning

The novelty of the proposed network is that it combines a memory network [10] with a recurrent neural network [11] to achieve highly efficient matching and robust substitution. Conventional networks are inefficient because they match the terms individually. However, the proposed network allows highly efficient matching by grouping multiple terms and processing them into memory networks. In addition, conventional networks cannot handle unknown words during substitution because substitution is rule-based. The proposed network can handle unknown words even in substitutions because it uses a recurrent neural network to perform substitution. The proposed network is a novel architecture that combines the advantages of both types of networks, i.e., the ability of memory networks to efficiently handle multiple data and the robust substitution capability of recurrent neural networks.

Studies have been conducted to integrate multiple knowledge bases, each described by different knowledge representations, to perform inference [12–14]. Our proposed network can support the integration of multiple knowledge bases with different knowledge representations because it can handle ambiguous data using pretrained word-distributed representations. As the proposed network can support the integration of multiple knowledge bases, we believe that it will contribute to improving the scalability of applications, such as chatbots, decision support systems (DSS), and online analytical processing (OLAP).

The proposed network was evaluated using two types of knowledge bases written in Prolog [15]. Furthermore, we compared the performance of the proposed network with that of baselines. Computational efficiency was evaluated by measuring the runtime, and robustness was evaluated by measuring the correct answer rate using data containing unknown words.

We first review related research in Sect. 2. In Sect. 3, symbolic processing is defined and described as the learning target. Section 4 describes robust symbolic processing using deep learning, which is a previous study on our proposed network. In Sect. 5, a network for unification using deep learning is proposed. Finally, Sect. 6 presents the experimental results for the proposed network.

2 Related work

2.1 Neuro-symbolic AI

Conventional symbolic processing has difficulty handling large amounts of ambiguous data and requires the construction of a strictly manual knowledge base. Those knowledge bases also make it difficult to answer queries containing unknown words. To address these problems, research has been conducted on Neuro-Symbolic AI, which combines symbolic processing and neural networks.

Prior to the emergence of deep learning, several studies were conducted on symbolic processing using neural networks [6, 7]. In addition to research on learning propositional logic [16–18], research has also been conducted on learning first-order predicate logic [19–21] and unification [22, 23]. However, these studies are limited to proposals for methods and their practicality was limited.

Since the emergence of deep learning, research has been conducted on symbolic processing using graph neural networks [8, 24–26] and recurrent neural networks [9, 27, 28]. Furthermore, research has been conducted on probabilistic logic programming using deep learning [29, 30]. Studies have also been conducted on answer set programming [31] and abductive learning using deep learning [32]. Research has been conducted on the unification of Prolog [8] using graph neural networks, probabilistic Prolog [33] using embedding vectors, Prolog [9] using recurrent neural

networks combined with embedding vectors, and unification of natural language [34] using a Transformer [35].

In the research using graph neural networks [8], unification has been achieved using end-to-end matching and substitution. However, the templates (e.g., $\#1(X, Y):-\#2(X, Z), \#3(Z, Y).$) must be created manually before the model is trained. Therefore, restrictions exist on the atomic formulae that can be handled. Furthermore, although the synonyms included in the training set were unified, unknown words that were not included in the training set could not be handled. In addition, a large knowledge base is inefficient because facts must be extracted from the knowledge base individually for matching and substitution.

In studies on probabilistic Prolog using embedding vectors [33], unification is achieved by extending probabilistic logic programming [36] with embedding vectors. As in [8], templates are required, which limits the range of atomic expressions that can be handled. Although the use of embedding vectors allows for synonym matching, it is inefficient when the size of the knowledge base is large because facts must be extracted individually from the knowledge base for matching. Furthermore, substitution is not realized through deep learning, so the substitution of synonyms is not supported.

In studies using recurrent neural networks combined with embedding vectors [9], unknown words not included in the training set can be unified using a word-distributed representation. In addition, unlike [8, 33], templates should not be provided in advance. Thus, the atomic formula has a high degree of freedom that can be handled. However, it is inefficient if the size of the knowledge base is large because facts must be extracted from the knowledge base individually for matching. Furthermore, matching was only realized using deep learning, and substitution was not realized using deep learning. Therefore, this does not support the substitution of unknown words.

In research using Transformers [34], it is necessary to extract facts individually from the knowledge base for matching, and substitution is not realized using deep learning. Furthermore, unification could not be performed if the data contained unknown words.

Furthermore, with the emergence of large language models (LLMs), such as ChatGPT in recent years, research efforts have been undertaken to integrate LLMs with Neuro-Symbolic AI [37–39]. Although LLMs can provide a natural interface using natural language, they can generate also hallucinations. To address this problem, a large symbolic knowledge base can be incorporated into LLMs to make advanced inferences. We believe that technologies, such as unification using Neuro-Symbolic AI, will be necessary to solve this problem in the future.

2.2 Memory networks

Conventionally, learning processing that requires memory, such as time-series data, is performed using recurrent neural networks. However, as the memory capacity of recurrent neural networks depends on the number of neurons, they are unsuitable for handling large amounts of data. Therefore, memory networks equipped with external memories have emerged to handle large amounts of data.

Studies have been conducted on memory networks that address the task of answering queries based on facts [10, 40]. Research has also been conducted to solve problems with graph structures by controlling readable and writable external memories using a controller composed of a neural network [41–43].

A differentiable neural computer (DNC) [42] is an extension of a neural turing machine (NTM) [41] that improves performance through an attention mechanism. The DNC uses an attention mechanism to calculate weights that indicate the addresses in the external memory that are read and written. Furthermore, research has been conducted on Seq-DNC-Seq [43], which is a network that connects a DNC to a decoder composed of long short-term memory (LSTM).

3 Symbolic processing

Here, we address symbolic processing based on a Prolog processing system. Prolog processing systems are used for reasoning and automated theorem proving in symbolic processing. When a Prolog processing system receives a query, it refers to the facts and rules stored in the knowledge base and infers an answer. A query comprises of one or more goals. A Prolog processing system infers.

goals using backward reasoning. Unification was necessary for this inference process.

Unification is the process of obtaining the same term by performing a substitution operation on both terms. When two terms are given, checking whether they are the same or whether they can be the same is known as matching. “Can be the same” means that two terms are the same after assigning objects to variables in the two terms. Statement 1 represents the term Tom is Bob’s father. If Statement 1 and another statement that is exactly the same exist, they match. Prolog treats names starting with uppercase letters as variables and lowercase letters as constants.

Therefore, proper nouns such as Tom are written in lowercase letters.

Statement 1: *father(tom, bob)*

In Statement 2, *X* is Bob’s father. Statements 1 and 2 can be unified. We can assign “*tom*” to the variable “*X*”. Therefore, the terms matched.

Statement 2: *father(X, bob)*

Assigning an object to a variable is called substitution. Substituting the variable “*X*” in Statement 2 with “*tom*” yields Statement 1. This is known as a substitution. Unification is achieved by combining matching and substitutions. In this study, matching is defined as the task of determining whether two terms are unifiable given two terms. Substitution is defined as the task of substituting two unifiable terms, given two terms, such that the two terms are the same.

Figure 1 illustrates an example of an inference made using Prolog. This shows the process of inference when a user asks the Prolog processing system the query “Who is Tom’s mother?” The goal of the inference shown in Fig. 1 is the query “Who is Tom’s mother?” as indicated in Statement 3.

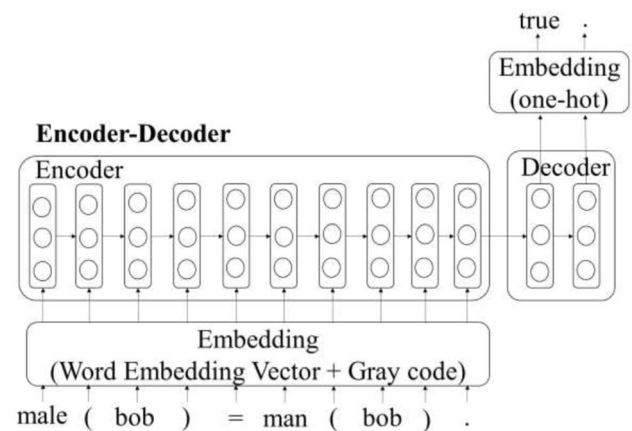
Statement 3: *mother(X, tom)*

First, the rule for Statement 4 was retrieved from the knowledge base. Statement 4 defines the rule that if *X* is female and *X* is *Y*’s parent, then *X* is *Y*’s mother. The left.

side of the rule is called the head and the right side is called the body. The head of Statement 4 is “*mother(X, Y)*” and the body is “*female(X), parent(X, Y)*”.

Statement 4: *mother(X, Y) : -female(X), parent(X, Y).*

Fig. 2 Existing robust deep learning-based symbolic processing for matching



in [9] it implies matching. Queries can contain variables. However, unlike the Prolog processing system, the model does not substitute for variables and determines whether matching is possible. For example, the model outputs Statement 15 instead of Statement 14 when Statement 13 is the input.

Statement 13: $male(X) = male(tom)$.

Statement 14: $X = tom$.

Statement 15: *true*.

Substitution is handled by processing outside the model after determining whether matching is possible.

When a query in the form of a word string is input into the matching model, the input embedding layer converts the string into a combined vector of word embeddings and gray code [44]. Word2Vec [45–47] is used to obtain word-embedding vectors. Word2Vec can obtain a vector representation of each word from large amounts of text data using neural networks. Common nouns of atoms such as “*male*” are word embeddings; however, logical symbols such as “(”, “)”, “,”, “.”, and proper nouns of atoms such as “*bob*” are encoded in the gray code. The gray code is characterized by a Hamming distance of one between adjacent codes. The reason for using the gray code is because the logical and constant symbols of proper nouns cannot be represented by word-embedding vectors. However, because word-embedding vectors are used to represent constant symbols, the model can output words that are not included in the training dataset.

Subsequently, the combined vector of the word embedding and gray code is passed to the encoder in the encoder-decoder model. The output from the decoder of the encoder-decoder model was embedded one-hot in the output embedding layer, and the matching word string was the output.

5 Proposed deep neural unification

This section proposes a network that efficiently executes the unification process described in Sect. 3 using deep learning. Furthermore, the proposed network enables efficient matching and substitution, even when unknown words are included. In addition, unlike in [8], templates need not be provided in advance. Thus, the atomic formula has a high degree of freedom that can be handled.

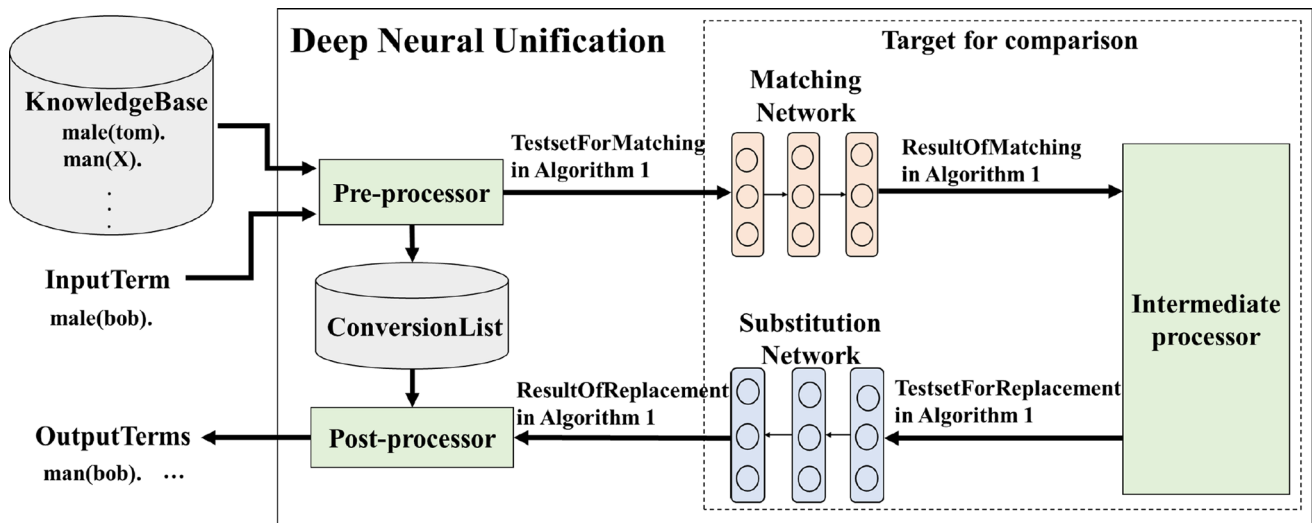


Fig. 3 Configuration of the proposed deep neural unification

5.1 Deep neural unification

We proposed an efficient unification network using matching and substitution networks. The matching networks realize matching of unification. The details of the matching networks are presented in Sect. 5.2. Substitution networks realize the substitution of unification. The details of the matching networks are presented in Sect. 5.3.

5.1.1 Configuration of proposed network

Figure 3 shows the configuration of the proposed network. *InputTerm* and *KnowledgeBase*, which are the targets of unification, are input to this network. *OutputTerms* is the result of substituting all facts that match *InputTerm* among the facts contained in *KnowledgeBase*.

Algorithm 1 Efficient deep neural unification algorithm

Require: *KnowledgeBase*, *InputTerm*
Ensure: *OutputTerms*

```

1: TestForMatching  $\leftarrow$  null
2: ConversionList  $\leftarrow$  null
3: NumberOfFacts  $\leftarrow$   $\alpha$ 
4: for  $i = 1$  to KnowledgeBase.length do
5:   if ( $i \% \text{NumberOfFacts} == 0$ ) then
6:     PreProcessor generates TestData by combining facts and
       InputTerm
7:     PreProcessor converts TestData to ConvertedTestData
8:     TestForMatching.append(ConvertedTestData)
9:     PreProcessor updates ConversionList
10:   end if
11: end for
12: ResultOfMatching  $\leftarrow$  null
13: for all TestData  $\leftarrow$  TestForMatching do
14:   Matching network predicts TestData
15:   Matching network stores the result in Result
16:   ResultOfMatching.append(Result)
17: end for
18: TestForSubstitution  $\leftarrow$  null
19: for all Result  $\leftarrow$  ResultOfMatching do
20:   IntermediateProcessor converts Result to ConvertedResult
21:   TestForSubstitution.append(ConvertedResult)
22: end for
23: ResultOfSubstitution  $\leftarrow$  null
24: for all TestData  $\leftarrow$  TestForSubstitution do
25:   Substitution network predicts TestData
26:   Substitution network stores the result in Result
27:   ResultOfSubstitution.append(Result)
28: end for
29: for all Result  $\leftarrow$  ResultOfSubstitution do
30:   PostProcessor converts Result to ConvertedResult
       with ConversionList
31:   OutputTerms.append(ConvertedResult)
32: end for
33: return OutputTerms

```

This network contains five modules: a preprocessor, matching network, intermediate processor, substitution network, and postprocessor. The preprocessor converts the *InputTerm* and *KnowledgeBase* into a format that can be input into the matching network. The matching network is a trained DNC [42], which is a memory network that performs matching. The intermediate processor converts the output of the matching network into a form that can be inputted into the substitution network. A substitution network is a trained recurrent neural network that performs substitutions. The postprocessor converts the output of the substitution network into *OutputTerms*.

The proposed network combines the advantages of two types of networks: the ability of memory networks to handle multiple data efficiently and the robust substitution capability of recurrent neural networks. A combination of two different networks can handle unification tasks more robustly and efficiently than a single network.

5.1.2 Efficient Deep Neural Unification Algorithm

Algorithm 1, entitled “Efficient Deep Neural Unification Algorithm,” presents the unification algorithm for the proposed network.

For example, suppose that Statement 16 is given as *InputTerm* and Statement 17 is stored in *KnowledgeBase*.

Statement 16: *male(bob)*.

Statement 17: *male(tom).man(X).female(X)male(bob)*.

The matching network enables Statement 18 to be extracted, which can be matched to Statement 16 using the terms in Statement 17.

Statement 18: *parent(mary, tom)*.

The substitution network substitutes Statement 16 and 18 to obtain Statement 19.

Statement 19: *male(bob).male(bob)*.

Finally, Statement 19 gives the output of *OutputTerms*.

The variable *TestForMatching* in line 1 of Algorithm 1 stores the test set input in the matching network. The variable *ConversionList* in line 2 of Algorithm 1 saves the results of converting the proper nouns included in *InputTerm* and *KnowledgeBase* into atoms for internal processing. The variable *NumberOfFacts* in line 3 of Algorithm 1 sets the number of facts in one input of the matching network. The proposed matching network can process matching by grouping multiple terms and setting a value for the variable *NumberOfFacts*. This enables the matching network to achieve efficient matching.

5.1.2.1 Preprocessor block The block in lines 4–11 of Algorithm 1 executes preprocessor processing when it receives *KnowledgeBase* denoted by Eq. (1). Figure 4 illustrates a flowchart of the process for this block. *InputTerm* denoted by *it*, and the variable *NumberOfFacts* denoted by α . f_i in the *KnowledgeBase* indicates fact.

$$\{f_i | i \in \mathbb{N}, 1 \leq i \leq \text{KnowledgeBase.length}\} \quad (1)$$

The preprocessor extracts facts from *KnowledgeBase* for each α and combines them with *it* to generate *TestData*. *TestData* is represented by Eq. (2).

$$\left\{ \left\{ it, f_{\alpha*j+i} | i \in \mathbb{N}, 1 \leq i \leq \alpha \right\} | j \in \mathbb{N}, 0 \leq j \leq \text{KnowledgeBase} \cdot \frac{\text{length}}{\alpha} \right\} \quad (2)$$

Furthermore, proper nouns included in the generated *TestData* were converted into atoms exclusively for internal processing. The *ConvertedTestData*, which is the value *TestForMatching*, is represented by Eq. (3).

$$\left\{ \left\{ it', f'_{\alpha*j+i} | i \in \mathbb{N}, 1 \leq i \leq \alpha \right\} | j \in \mathbb{N}, 0 \leq j \leq \text{KnowledgeBase} \cdot \frac{\text{length}}{\alpha} \right\} \quad (3)$$

For example, converting the proper nouns in Statement 20 yields Statement 21.

Statement 20: *male(bob).male(tom).man(X).female(X)*.

Statement 21: *male(p1).male(p2).man(X).female(X)*.

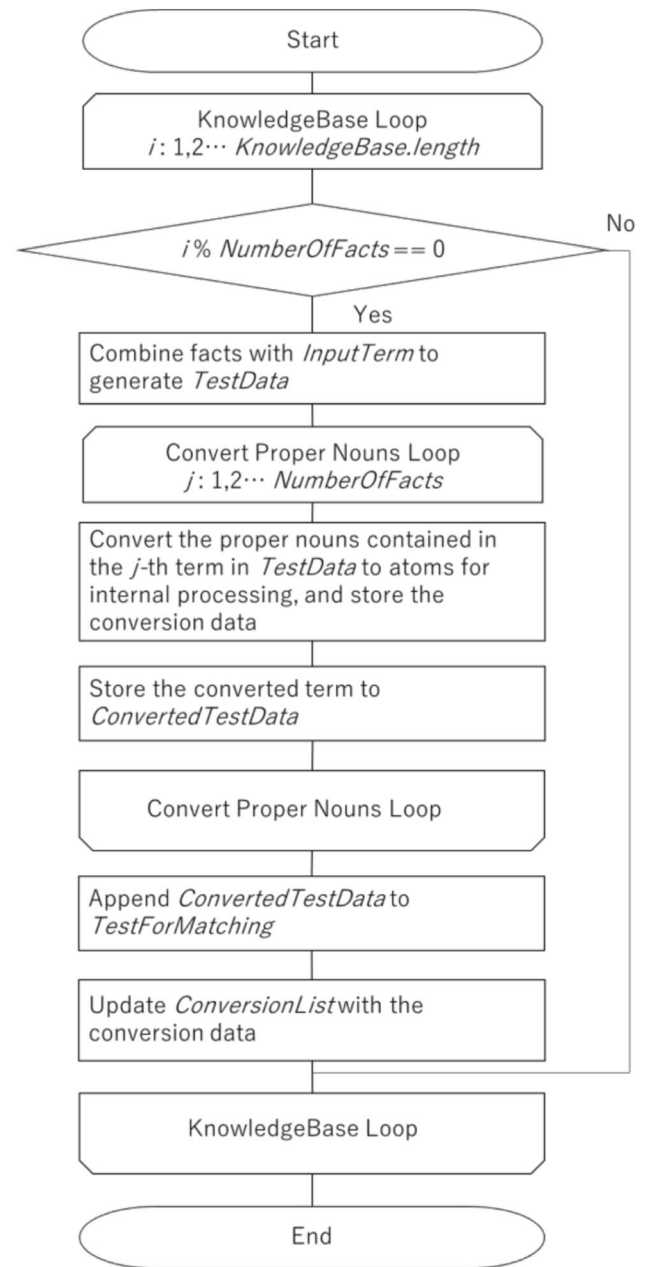
The variable *ConversionList* stores a list indicating that “bob” has been converted to “p1” and “tom” to “p2”. As.

another example, converting the proper nouns in Statement 22 yields Statement 23.

Statement 22: *female(mary).woman(X).female(X).male(bob)*.

Statement 23: *female(p1).woman(X).female(X).male(p2)*.

ConversionList stores a list indicating that “mary” has been converted to “p1” and “bob” to “p2.” *ConversionList* is denoted by Eq. (4).

Fig. 4 Flowchart of a preprocessor

$$\left\{ \left\{ \left(v_{ij}, v'_{ij} \right) \mid i \in \mathbb{N}, 0 \leq i \leq \text{KnowledgeBase} \cdot \frac{\text{length}}{\alpha} \right\} \mid j \in \mathbb{N}, 0 \leq j \leq v_i.\text{length} \right\} \quad (4)$$

ConversionList contains as many pairs of lists as the test data. A pair of lists indicates the values before and after conversion. This means that the value v_{ij} is converted to v'_{ij} . By exclusively converting proper nouns into atoms for internal processing, the types of words learned by matching and substitution networks can be reduced, thereby enabling learning with less data. The output of the preprocessor is stored in the variable *TestForMatching*.

5.1.2.2 Matching network block Variable *ResultOfMatching* in line 12 of Algorithm 1 stores the prediction result of the matching network. The block in lines 13–17 of Algorithm 1 performs matching network processing when it receives the variable *TestForMatching*. The variable *Result*, which is the output of the matching network, is added to the variable *ResultOfMatching*. *ResultOfMatching* is denoted by Eq. (5).

$$\left\{ \{it', rm_{\alpha*j+i} | i \in \mathbb{N}, 1 \leq i \leq \alpha\} | j \in \mathbb{N}, 0 \leq j \leq KnowledgeBase.length/\alpha \right\} \quad (5)$$

$rm_{\alpha*j+i}$ is the matching result for it' . $rm_{\alpha*j+i}$ is the same value as $f'_{\alpha*j+i}$ if it matches with it' , or a string replaced by “_” if it does not match.

5.1.2.3 Intermediate processor block The variable *TestForSubstitution* in line 18 of Algorithm 1 stores the test set as an input into the substitution network. The block in lines 19–22 of Algorithm 1 executes the processing of the intermediate processor when it receives variable *ResultOfMatching*. Figure 5 depicts a flowchart of the process for this block. The intermediate processor converts variable *ResultOfMatching* into a form.

that can be input into the substitution network. *TestForSubstitution* is expressed by Eq. (6).

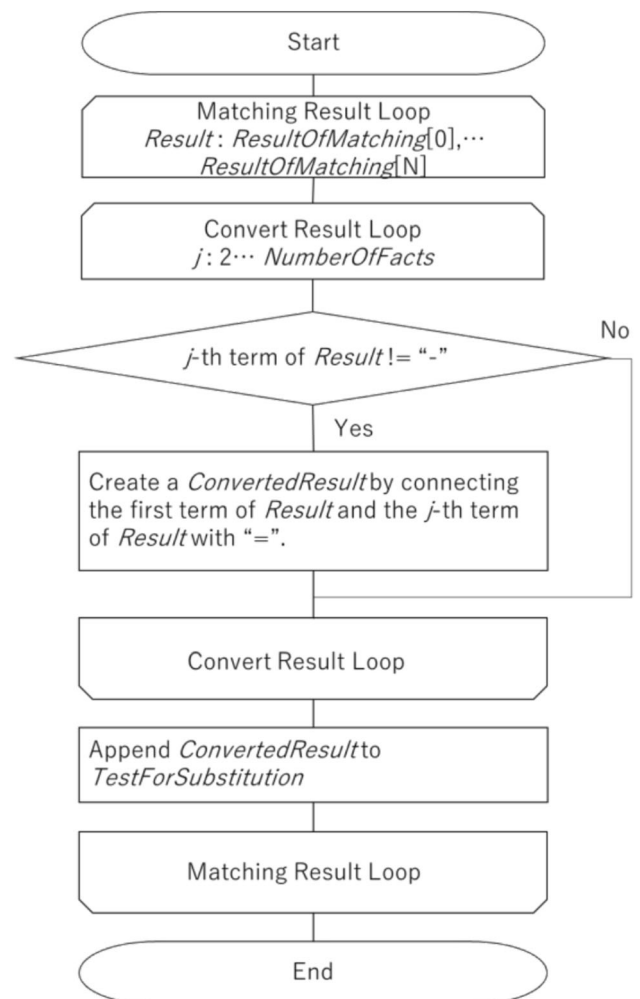
$$\left\{ \{it' = rm'_{\alpha*j+i} | i \in \mathbb{N}, 1 \leq i \leq \alpha\} | j \in \mathbb{N}, 0 \leq j \leq KnowledgeBase.length/\alpha, rm'_{\alpha*j+i} \text{ is a different value from " _" in } rm_{\alpha*j+i} \right\} \quad (6)$$

$rm'_{\alpha*j+i}$ is the result of a successful match with it' . For example, if Statement 24 is input into the matching network and Statement 25 is obtained, Statement 25 is converted into Statement 26.

Statement 24: $male(p1).male(p2).man(X).female(X)$.

Statement 25: $male(p1)._male(X)._Statement 26: male(p1) = man(X)$.

Fig. 5 Flowchart of an intermediate processor



Only terms matched by the matching network are input into the substitution network.

5.1.2.4 Substitution network block Variable *ResultOfSubstitution* in line 23 of Algorithm 1 stores the prediction results for the substitution network. The block in lines 24–28 of Algorithm 1 performs substitution network processing when it receives the variable *TestForSubstitution*. The variable *Result*, which is the output of the substitution network, is added to the variable *ResultOfSubstitution*. *ResultOfSubstitution* is expressed by Eq. (7).

$$\{\{rs_{ij}|i \in \mathbb{N}, 0 \leq i \leq \text{TestForSubstitution.length}\}|j \in \mathbb{N}, 0 \leq j \leq \text{TestForSubstitution}_i.\text{length}\} \quad (7)$$

rs_{ij} denotes the substitution results.

5.1.2.5 Post processor block The block in lines 29–31 of Algorithm 1 receives the variables *ResultOfSubstitution* and *ConversionList* and executes postprocessor processing. Figure 6 shows a flowchart of the process for this block. The postprocessor refers to the variable *ConversionList* and converts the atoms into the original proper nouns exclusively for internal processing. For example, Statement 27 can be converted into Statement 28.

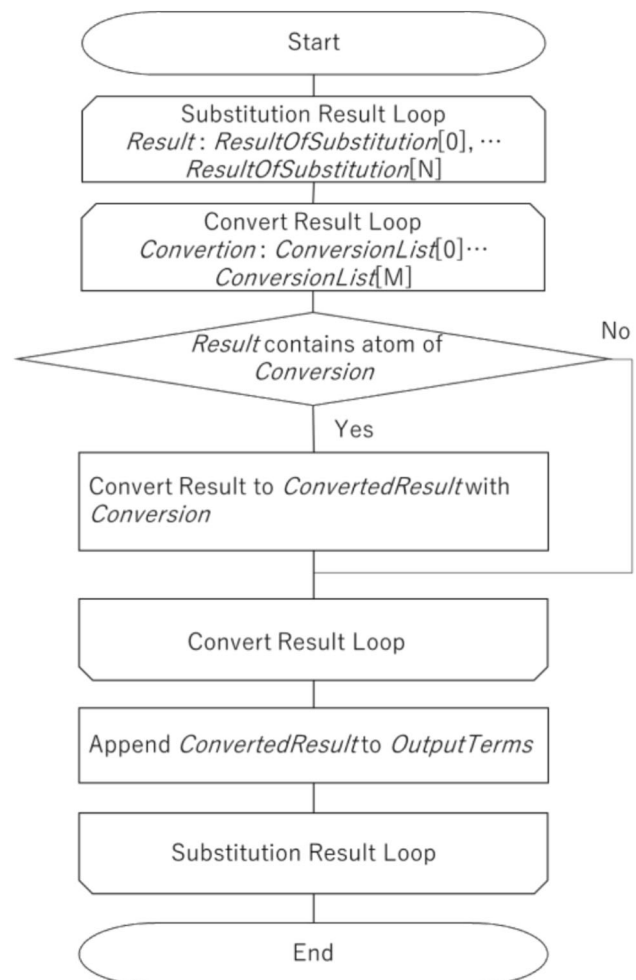
Statement 27: *male(p1)*

Statement 28: *male(bob)*

Finally, the postprocessor output is stored as a variable *OutputTerms*. *OutputTerms* is denoted by Eq. (8).

$$\{\{ot_{ij}|i \in \mathbb{N}, 0 \leq i \leq \text{ResultOfSubstitution.length}\}|j \in \mathbb{N}, 0 \leq j \leq \text{ResultOfSubstitution}_i.\text{length}\} \quad (8)$$

Fig. 6 Flowchart of a postprocessor



ot_{ij} denotes the inverse conversion of rs_{ij} with respect to *ConversionList*.

5.2 Matching networks

Figure 7 shows the network that realizes the matching of the unification used in our proposed network. In the matching model in [9], facts are extracted and matched individually from the knowledge base, whereas the matching networks in the proposed network can simultaneously match multiple facts. The matching model in [9] uses a recurrent neural network, whereas the proposed matching network uses the DNC [42], a memory network. It is difficult for recurrent neural networks to handle large amounts of data because the memory capacity of their networks depends on the number of neurons. By contrast, DNC can easily store large amounts of data and match multiple facts simultaneously because they have external memory. Compared to memory networks such as NTM, the DNC has a higher ability to learn the relationships among memories for continuous data and series and handle complex data structures. Therefore, we decided to use the DNC as the memory network in our proposed network.

As described in Sect. 5.1, the input to the matching network represented by Eq. (9), is concatenated as a string.

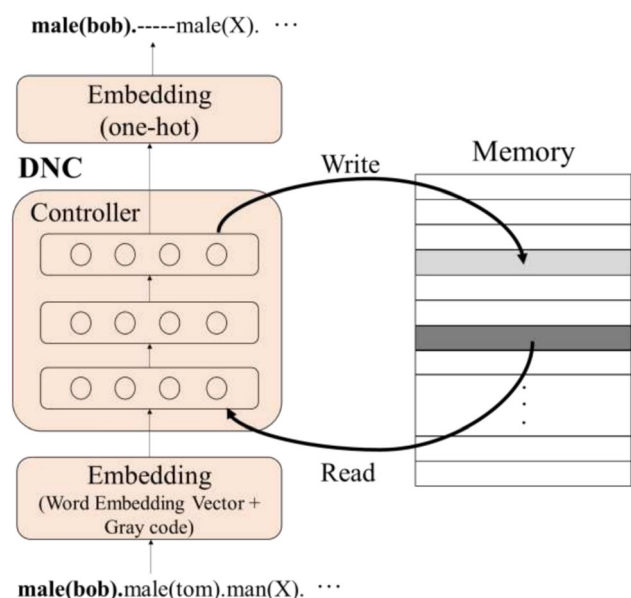
$$\{it', f'_i | i \in \mathbb{N}, 1 \leq i \leq \alpha\} \quad (9)$$

it' denotes the input for unification and f'_i is a fact contained in the knowledge base. α indicates the number of facts in one input in the matching network. The input of the proposed matching network was converted into a combined vector of word embeddings and gray codes using the input-embedding layer. Once the input-embedding layer receives an input, each token of the input is transformed into a combined vector $input_k$ of $word_k$, which is embedded by a word-embedding vector, and $graycode_t$, which is embedded by the gray code, as shown in Eq. (10). Here, k denotes the input index.

$$input_k = [word_k; graycode_k] \quad (10)$$

Common nouns of atoms are word embeddings, and logical symbols, and the proper nouns of atoms are encoded in gray code. Those that were not included in the pretrained word embeddings were considered proper nouns, and those that were included were considered common nouns. This automatically discriminates between proper and common nouns.

Fig. 7 Matching networks in the proposed network



For word embedding in the input embedding layer, a model based on Word2Vec [45–47] called GoogleNews-vectors-negative300 was used. GoogleNews-vectors-negative300 was pretrained using a dataset of approximately 100 billion words. The model contains 300-dimensional vectors for 3 million words and phrases.

The DNC controller controls the read and write heads to read and write to external memory. The controller comprises an LSTM [48], which is a type of recurrent neural network. Controller processing can be expressed using Eq. (11), where h_t represents the hidden state at time step t , c_t is the cell state at time step t , and r_{t-1} represents the vector read from external memory in step $t-1$.

$$h_t, c_t = LSTM([input_t, r_{t-1}], h_{t-1}, c_{t-1}) \quad (11)$$

Equation (12) is the interface vector ξ_t . ξ_t is computed using h_t and a weight matrix of memory operation parameters W_ξ . The interface vector combines the write key k_t^w , write strength β_t^w , write vector v_t , erase vector e_t , read key k_t^r , and read strength β_t^r required to read and write external memory.

$$\xi_t = W_\xi h_t \quad (12)$$

The weights w_t^w that determine where and how much the controller writes to external memory are calculated in Eq. (13). M_{t-1} is the state of external memory at step $t-1$. *cosim* refers to cosine similarity.

$$w_t^w = \text{softmax}(\beta_t^w \text{cosim}(M_{t-1}, k_t^w)) \quad (13)$$

The rewritten external memory is shown in Eq. (14).

$$M_t = M_{t-1} \odot (1 - w_t^w e_t^T) + w_t^w v_t^T \quad (14)$$

The weights w_t^r that determine where the controller reads to external memory are calculated using Eq. (15).

$$w_t^r = \text{softmax}(\beta_t^r \text{cosim}(M_t, k_t^r)) \quad (15)$$

The vector read out is as in Eq. (16).

$$r_t = M_t^T w_t^r \quad (16)$$

The final output $\hat{y}_t$ is expressed as Eq. (17), where W_y represents the weight matrix used to compute the final output. $\hat{y}_t$ represents an estimate of a one-hot vector embedded in the output-embedding layer.

$$\hat{y}_t = \text{softmax}(W_y h_t) \quad (17)$$

The one-hot vector enables output even if unknown words are included in the input, similar to the matching model in [9]. As described in Sect. 5.1, the matching network output is represented by Eq. (18) and concatenated as a string. rm_i is the matching result for it' ; rm_i is the same value as f_i' if it matches with it' , or a string replaced by “_” if it does not match.

$$\{it', rm_i | i \in \mathbb{N}, 1 \leq i \leq \alpha\} \quad (18)$$

Cross entropy is used as the loss function. The *Loss* between the model's prediction $\hat{y}_t$ and the actual value y_t is calculated using Eq. (19).

$$Loss = - \sum_{t=1}^T y_t \log(\hat{y}_t) \quad (19)$$

In the matching network, the number of reading heads was set to four, and RMSProp [49] was used for optimization. The learning rate of RMSProp was set to 10^{-4} , with a momentum of 0.9. The training of the matching network was repeated for 10,000 epochs with a batch size of one. These hyperparameters were determined using a grid search when the number of facts in an input was five.

Finally, the matching network learns to output Statement 30 when Statement 29 is used as input.

Statement 29: *male(bob).male(tom).man(X).female(X).*

Statement 30: *male(bob)._____male(X)._____*

In Statement 31, the first terms in Statements 29 and 30 are the targets of unification with the knowledge base facts.

Statement 31: *male(bob).*

The second and subsequent terms in Statement 29 are enumerate the facts in the knowledge base. In the example in Statement 29, only four facts are listed; however, more can be listed. The second and subsequent terms in Statement 30 correspond to the second and subsequent terms in Statement 29 and are substituted with “-” except in Statement 32, which matches Statement 31.

Statement 32: *man(X).*

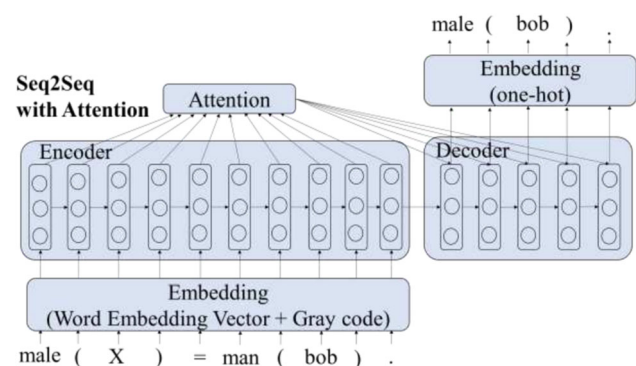
Here, because “*man*” in Statement 32 is an unknown word, in Statement 30, it is substituted with “*male*”, which is known as the matching network, and output. The DNC is suitable for matching a large amount of data but is not suitable for handling program synthesis, such as substitution.

5.3 Substitution networks

Figure 8 shows a network that realizes the substitution of unification used in our proposed network. The proposed substitution network used an encoder-decoder model. Therefore, we used an encoder-decoder model suitable for handling program synthesis in substitution networks. Substitution can also be regarded as a program-synthesis process. In [9], a performance comparison experiment between Seq2Seq with attention [50, 51] and a Transformer [35] was conducted for symbolic processing. The results of the experiment, Seq2Seq with attention performed better than Transformer. Therefore, we used Seq2Seq with attention for the substitution network.

As described in Sect. 5.1, the input of the substitution network is represented by Eq. (20).

Fig. 8 Substitution networks in the proposed network



$$it' = rm'_i \quad (20)$$

rm'_i is the result of a successful match with it' and it' is the input for unification. The input of the proposed substitution network is converted into a combined vector of word embeddings and gray code using the input-embedding layer. Once the input-embedding layer receives an input, each token of the input is transformed into a combined vector $input_k$ of $word_k$, which is embedded by a word-embedding vector, and $graycode_k$, which is embedded by the gray code, as shown in Eq. (21). k is the input index.

$$input_k = [word_k; graycode_k] \quad (21)$$

Common nouns of atoms are word embeddings, and logical symbols and proper nouns of atoms are encoded in gray code. Those not included in the pretrained word embeddings are considered proper nouns, and those included are considered common nouns. This automatically discriminates between proper and common nouns. For word embedding in the input-embedding layer, GoogleNews-vectors-negative300 is used.

Seq2Seq with Attention was applied to the encoder-decoder model. The encoder contains three layers of Bi-LSTM. The processing $LSTM_n^{fwd}$ of the forward LSTM of the n^{th} layer in the Bi-LSTM can be expressed as in Eq. (22), and the processing $LSTM_n^{bwd}$ of the reverse LSTM of the n^{th} layer in the Bi-LSTM, as in Eq. (23), where $h_t^{enc(n)}$ is the hidden state at time step t in the n^{th} layer and $c_t^{enc(n)}$ is the cell state at time step t in the n^{th} layer. The right and left arrows in the vector indicate forward and reverse calculations, respectively.

$$\begin{matrix} \rightarrow \\ h_t \end{matrix}^{enc(n)}, \begin{matrix} \rightarrow \\ c_t \end{matrix}^{enc(n)} = LSTM_n^{fwd} \left(\begin{matrix} \rightarrow \\ h_{t-1} \end{matrix}^{enc(n)}, \begin{matrix} \rightarrow \\ c_{t-1} \end{matrix}^{enc(n)}, x_t \right) \quad (22)$$

$$\begin{matrix} \leftarrow \\ h_t \end{matrix}^{enc(n)}, \begin{matrix} \leftarrow \\ c_t \end{matrix}^{enc(n)} = LSTM_n^{bwd} \left(\begin{matrix} \leftarrow \\ h_{t-1} \end{matrix}^{enc(n)}, \begin{matrix} \leftarrow \\ c_{t-1} \end{matrix}^{enc(n)}, x_t \right) \quad (23)$$

The output of the n^{th} LSTM layer is expressed by Eq. (24).

$$h^{enc(n)} = \left[\begin{matrix} \rightarrow \\ h_t \end{matrix}^{enc(n)}; \begin{matrix} \leftarrow \\ h_t \end{matrix}^{enc(n)} \right] \quad (24)$$

The final output of the encoder is given by Eq. (25), where T denotes the length of the input sequence.

$$H = \{h_1^{enc(3)}, h_2^{enc(3)}, \dots, h_T^{enc(3)}\} \quad (25)$$

For the attention mechanism, $Score_{ti}$ was calculated using Eq. (26), using Additive Attention at each step of the decoder. where h_t^{dec} is the current hidden state of the decoder, h_i^{enc} the encoder output vector, W_a represents the weight matrix applied to hidden state of the decoder, U_a is the weight matrix applied to the encoder output vector, and v_a is the weight vector used to compute the score. In addition, t is the time step of the decoder and i is the time step of the encoder. The parameters related to attention are marked with subscript a .

$$Score_{ti} = v_a^T \cdot \maxout(W_a h_t^{dec} + U_a h_i^{enc}) \quad (26)$$

Using $Score_{ti}$, the Attention weight α_{ti} is calculated as in Eq. (27).

$$\alpha_{ii} = \frac{\exp(\text{Score}_{ii})}{\sum_{j=1}^T \exp(\text{Score}_{ij})} \quad (27)$$

Context_t , which is the context vector, is calculated using Eq. (28).

$$\text{Context}_t = \sum_{i=1}^T \alpha_{ii} h_i^{\text{enc}} \quad (28)$$

The decoder consists of a one-layer LSTM, and the LSTM processing is expressed in Eq. (29). where h_t^{dec} is the hidden state at time step t and c_t^{dec} is the cell state at time step t .

$$h_t^{\text{dec}}, c_t^{\text{dec}} = \text{LSTM}(h_{t-1}^{\text{dec}}, c_{t-1}^{\text{dec}}, y_{t-1}, \text{Context}_t) \quad (29)$$

The final output $\hat{y}_t$ of the decoder is expressed by Eq. (30). where W_o is the weight matrix used to compute the output of the final decoder. $\hat{y}_t$ is an estimate of a one-hot vector embedded in the output-embedding layer.

$$\hat{y}_t = \text{softmax}(W_o h_t^{\text{dec}}) \quad (30)$$

The one-hot vector enables output even if unknown words are included in the input, similar to the matching model in [9]. As described in Sect. 5.1, the output of the substitution network is represented in the form rs_i . rs_i denotes the substitution results.

Cross entropy is used as the loss function. The *Loss* between the model prediction $\hat{y}_t$ and the actual value y_t is calculated using Eq. (31).

$$\text{Loss} = - \sum_{t=1}^T y_t \log(\hat{y}_t) \quad (31)$$

Adam [52] was used to optimize the substitution network. The learning rate of Adam was set to 10^{-3} , β_1 to 0.9, and β_2 to 0.999. The batch size was set to 128, and the dropout was set to 0.1 for both the encoder and decoder. Training of the substitution network was repeated for five epochs. These hyperparameters were determined using a grid search.

Finally, the substitution network learns to output Statement 34 when Statement 33 is used as input.

Statement 33: $\text{male}(\text{bob}) = \text{man}(X)$.

Statement 34: $\text{male}(\text{bob})$.

The left side of Statement 33 states the fact of the knowledge base and target are unified, whereas the right side represents the fact of the knowledge base that matches the left side. Statement 34 is the result of substituting the left and right sides of Statement 33. Here, as “*man*” in Statement 33 is an unknown word, “*male*” is output in Statement 34, which is known to the substitution network.

A substitution network differs from a matching network because it processes knowledge-based facts individually. However, because the substitution network only substitutes facts matched by the matching network, no efficiency loss occurs.

6 Evaluation experiments

We trained the proposed network and conducted evaluation experiments using two types of knowledge bases written in Prolog. Here, we first present the experimental results using two types of knowledge bases, and then discuss the results.

6.1 Experiments using Kinsources

We conduct an experiment using Kinsources [53] as the knowledge base. Kinsources is a collection of data representing kinship relationships written in Prolog and consists of 5,887 atoms and 10 predicates. Statements 35 and 36 are examples of facts contained in Kinsources knowledge base. Statement 35 indicates that “*p3876*” is the father of “*p3877*”. “*p3876*” and “*p3877*” are proper nouns that represent person names. Statement 36 indicates that “*p3428*” is a female. “*p3428*” is also a proper noun that represents the name of a person.

Statement 35:*father(p3876,p3877)*.

Statement 36:*female(p3428)*.

Table 1 presents the datasets obtained using Kinsources. Facts were extracted uniformly at random from Kinsources and some arguments in these facts were variablized uniformly at random. By inputting them into the preprocessor, as shown in Fig. 3, the dataset listed in Table 1 was generated. The training and test sets for known words contain predicates such as “*father*,” “*mother*,” “*male*,” and “*female*,” which originally existed in the Kinsources knowledge base. However, the test set for unknown words only included predicates that are not included in the training set. The test set for unknown words (4 words) includes predicates “*dad*,” “*mom*,” “*man*,” and “*woman*”. The test set for unknown words (12 words) includes predicates “*dad*,” “*daddy*,” “*papa*,” “*mom*,” “*mama*,” “*mum*,” “*man*,” “*males*,” “*Male*,” “*woman*,” “*females*,” and “*Female*”. Furthermore, we created separate datasets with 5, 20, and 100 facts included in each input to the network. In word-distributed representations, lowercase and uppercase letters and singular and plural forms are distinguished, and all unknown words are distinguished as different vectors.

However, if the general knowledge base is used as it is, the probability of a matching fact appearing in a certain.

term is extremely low. Therefore, we created a dataset with a matching rate of approximately 11–13% for learning and evaluation. This technique is called undersampling and allows for the training of unbalanced data. In addition, the number of atoms used for internal processing substituted with the preprocessor, as shown in Fig. 3, was limited to five types.

Table 2 lists the correct answer rates for the unification using the dataset in Table 1. The correct answer rate of known words is the ratio of the exact correct unification results obtained using the test set for known words and is calculated using Eq. (32).

Table 1 Dataset obtained using Kinsources. Kinsources is an accessible database that represents kinship relationships written in Prolog. There are three types of datasets with 5, 10, and 100 facts in one test data

Number of facts in one test data ¹	5	20	100
Training Set (Rate of facts that can be matching ²)	267,417 (0.1247)	267,393 (0.1248)	267,361 (0.1249)
Test Set for known words (Rate of facts that can be matching ²)	33,426 (0.1250)	33,423 (0.1255)	33,419 (0.1252)
Test Set for unknown words (4 words) ³ (Rate of facts that can be matching ²)	16,656 (0.1114)	16,707 (0.1140)	16,738 (0.1143)
Test Set for unknown words (12 words) ⁴ (Rate of facts that can be matching ²)	16,656 (0.1114)	16,707 (0.1140)	16,738 (0.1143)

¹ $\propto$ in Algorithm 1, the number of facts input into the network at one time. ²Probability that a fact matching a given term will appear. ³Includes four unknown predicates. ⁴Includes 12 unknown predicates

Table 2 Correct answer rates for unification using the dataset in Table 1. This table presents the result of the correct answer rate using three different datasets where the number of facts in one test data is 5, 20, and 100

Number of facts in one test data ¹		5	20	100
Prolog system	Known words ²	1.0000	1.0000	1.0000
	Unknown words (4 words) ³	0.0000	0.0000	0.0000
	Unknown words (12 words) ⁴	0.0000	0.0000	0.0000
DNC	Known words ²	0.6768	0.3648	0.1353
	Unknown words (4 words) ³	0.2913	0.0062	0.0007
	Unknown words (12 words) ⁴	0.4410	0.2608	0.1049
Seq-DNC-Seq	Known words ²	0.5769	0.2093	0.0001
	Unknown words (4 words) ³	0.2305	0.0244	0.0000
	Unknown words (12 words)	0.3573	0.2883	0.0001
Seq2Seq with Attention	Known words ²	0.9987	0.9048	Not executable
	Unknown words (4 words) ³	0.4978	0.4232	Not executable
	Unknown words (12 words) ⁴	0.7523	0.4734	Not executable
Seq2Seq + Seq2Seq	Known words ²	0.8708	0.9475	Not executable
	Unknown words (4 words) ³	0.4401	0.4761	Not executable
	Unknown words (12 words)	0.6405	0.5791	Not executable
Proposed network	Known words ²	1.0000	1.0000	1.0000
	Unknown words (4 words) ³	0.4980	0.4993	0.5025
	Unknown words (12 words) ⁴	0.7277	0.7407	0.7088

¹ α in Algorithm 1, the number of facts input into the network at one time. ²The results using Test Set for known words. ³The results using Test Set for unknown words (4 words). ⁴The results using Test Set for unknown words (12 words)

The values highlighted in bold in the tables indicate the best performance among all compared methods for each respective metric

$$\text{Correct Answer Rate of known words} = \frac{\text{Num of exact correct unification result}}{\text{Num of test set for known words}} \quad (32)$$

Incorrect answers included cases where the syntax was correct, but the unification result was incorrect, and cases where the unification result was correct but the syntax was.

incorrect (e.g., a period is missing). The correct answer rate was calculated using the exact correct answer when both the syntax and unification results were correct. The correct result was the most general unification. If the syntax in the output is incorrect, the proposed network outputs a result in which unification is not possible.

The correct answer rate of unknown words is the ratio of the exact correct unification results obtained using the test set for unknown words and is calculated using Eq. (33).

$$\text{Correct Answer Rate of known words} = \frac{\text{Num of exact correct unification result}}{\text{Num of testset for known words}} \quad (33)$$

The correct answer rate of unknown words was calculated in the same manner as the correct answer rate of known words. Incorrect answers included cases where the syntax was correct but the unification result was incorrect and cases where the unification result was correct but the syntax was incorrect.

The matching and substitution networks of the proposed network are trained using the training sets listed in Table 1. GoogleNews-vectors-negative300, which was used for word embedding in each network, was pre-trained. The hyperparameters of the proposed network were determined using a grid search when the number of facts in an input was five.

In addition to our proposed network, we performed experiments by replacing the “Target for comparison” in Fig. 3 with the Prolog system, DNC [42], Seq-DNC-Seq [43],

Seq2Seq with attention [51], and Seq2Seq + Seq2Seq as a baseline. The input layers of the DNC and Seq-DNC-Seq were modified to input a combined vector of word embedding and gray code. The output layers were modified to generate one-hot vectors. Seq2Seq with attention uses the model proposed in [9]. Seq2Seq + Seq2Seq replaces the matching network of the proposed network with that of Seq2Seq with attention. All word embeddings used the pretrained GoogleNews-vectors-negative300. These hyperparameters of these baselines were determined using grid search when the number of facts in an input was five.

6.2 Experiments using IMDb

We conducted an experiment using IMDb [54] as the knowledge base. IMDb is a movie and television-related database written in kLog derived from Prolog and contains 20,658 atoms and 14 types of predicates. In this experiment, IMDb was converted from kLog to the Prolog format.

Statements 37 and 38 are examples of facts from the IMDb knowledge base. Statement 37 indicates that “*p1527818*” is the actor of “*m625590*.” “*p1527818*” is a proper noun that represents the name of a person. “*m625590*” is a proper noun that represents the name of a movie. Statement 38 indicates that “*p2050796*” represents a person. “*p2050796*” is also a proper noun that represents the name of a person.

Statement 37: *actor(p1527818, m625590)*.

Statement 38: *person(p2050796)*.

Table 3 presents the dataset obtained using IMDb. Facts were extracted uniformly at random from the IMDb, and some arguments in these facts were variabilized uniformly at random. The dataset listed in Table 3 was generated by inputting data into the preprocessor, as shown in Fig. 3. The training and test sets for known words contain predicates such as “*person*,” “*company*,” “*movie*,” and “*actor*” which originally existed in the IMDb knowledge base. However, the test set for unknown words only included predicates that were not included in the training set. The test set for unknown words (4 words) includes predicates “*someone*,” “*firm*,” “*film*,” and “*actress*”. The test set for unknown words (12 words) includes predicates “*someone*,” “*somebody*,” “*anyone*,” “*firm*,” “*corporation*,” “*manufacture*,” “*film*,” “*cinema*,” “*flick*,” “*actress*,” “*thespian*,” and “*thesp*.” Furthermore, we created separate datasets with 5, 20, and 100 facts included in each input into the network.

We created a dataset with a matching rate of 12–14% for learning and evaluation. In addition, the number of atoms used for internal processing substituted with the preprocessor, as shown in Fig. 3, was limited to five types.

Table 4 lists the correct answer rates for the unification using the dataset in Table 3. The matching and substitution networks in the proposed network were trained using the training sets listed in Table 3. GoogleNews-vectors-negative300, which was used for word embedding in each network, was pretrained. These hyperparameters of the proposed network were determined using grid search when the number of facts in an input was five.

Table 3 Dataset obtained using IMDb. IMDb is an accessible movie and television-related database written in kLog derived from Prolog. There are three types of datasets with 5, 10, and 100 facts in one test data

Number of facts in one test data ¹	5	20	100
Training Set	171,175	171,175	171,169
(Rate of facts that can be matching ²)	(0.1425)	(0.1429)	(0.1431)
Test Set for known words	21,396	21,396	21,395
(Rate of facts that can be matching ²)	(0.429)	(0.1421)	(0.1422)
Test Set for unknown words (4 words) ³	12,172	12,293	12,270
(Rate of facts that can be matching ²)	(0.1202)	(0.1210)	(0.1225)
Test Set for unknown words (12 words) ⁴	12,172	12,293	12,270
(Rate of facts that can be matching ²)	(0.1202)	(0.1210)	(0.1225)

¹ $\propto$ In Algorithm 1, the number of facts input into the network at one time. ²The probability that a fact matching a given term will appear. ³Include four unknown predicates. ⁴Include 12 unknown predicates

Table 4 Correct answer rates for unification using the dataset in Table 3. This table presents the results of the correct answer rate using three different datasets where the number of facts in one test data is 5, 20, and 100

Number of facts in one test data ¹		5	20	100
Prolog system	Known words ²	1.0000	1.0000	1.0000
	Unknown words (4 words) ³	0.0000	0.0000	0.0000
	Unknown words (12 words) ⁴	0.0000	0.0000	0.0000
DNC	Known words ²	0.7335	0.4649	0.2103
	Unknown words (4 words) ³	0.5664	0.6591	0.3170
	Unknown words (12 words) ⁴	0.7658	0.6341	0.2049
Seq-DNC-Seq	Known words ²	0.6018	0.4391	0.0193
	Unknown words (4 words) ³	0.5887	0.5965	0.0181
	Unknown words (12 words) ⁴	0.6057	0.6867	0.0765
Seq2Seq with Attention	Known words ²	0.9991	0.9350	Not executable
	Unknown words (4 words) ³	0.9984	0.5285	Not executable
	Unknown words (12 words) ⁴	0.9964	0.5145	Not executable
Seq2Seq + Seq2Seq	Known words ²	0.9992	0.3189	Not executable
	Unknown words (4 words) ³	0.9992	0.4407	Not executable
	Unknown words (12 words) ⁴	0.9367	0.4286	Not executable
Proposed network	Known words ²	0.9881	1.0000	1.0000
	Unknown words (4 words) ³	0.9966	0.9252	0.9965
	Unknown words (12 words) ⁴	0.9771	0.9785	0.9985

¹ α In Algorithm 1, the number of facts input into the network at one time. ²The results using Test Set for known words. ³The results using Test Set for unknown words (4 words). ⁴The results using Test Set for unknown words (12 words)

The values highlighted in bold in the tables indicate the best performance among all compared methods for each respective metric

In addition to our proposed network, we performed experiments by replacing the “Target for comparison” in Fig. 3 with the Prolog system, the DNC [42], Seq-DNC-Seq [43], Seq2Seq with attention [51], and Seq2Seq + Seq2Seq as a baseline. The input layers of DNC and Seq-DNC-Seq were modified to input a combined vector of word embedding and gray code. The output layers were modified to generate one-hot vectors. Seq2Seq with attention uses the model proposed in [9]. Seq2Seq + Seq2Seq replaces the matching network of the proposed network with Seq2Seq with attention. All word embeddings used the pretrained GoogleNews-vectors-negative300. The hyperparameters of these baselines were determined using grid search when the number of facts in an input was five.

Table 5 Wilcoxon signed-rank sum test results ($p < 0.05$). The correct answer rates included in Tables 2 and 4 were used in the calculations

Number of facts in one test data ¹	5	20	100	All (5, 20, 100) ²
Proposed network VS Prolog system	$p = 0.075$	$p = \mathbf{0.027}$	$p = \mathbf{0.027}$	$p = \mathbf{0.00052}$
Proposed network VS DNC	$p = \mathbf{0.028}$	$p = \mathbf{0.028}$	$p = \mathbf{0.028}$	$p = \mathbf{0.00020}$
Proposed network VS Seq-DNC-Seq	$p = \mathbf{0.028}$	$p = \mathbf{0.028}$	$p = \mathbf{0.028}$	$p = \mathbf{0.00020}$
Proposed network VS Seq2Seq with Attention	$p = 0.12$	$p = \mathbf{0.028}$	$p = \mathbf{0.027}$	$p = \mathbf{0.0032}$
Proposed network VS Seq2Seq + Seq2Seq	$p = 0.12$	$p = \mathbf{0.028}$	$p = \mathbf{0.027}$	$p = \mathbf{0.00033}$

¹ α in Algorithm 1, the number of facts input into the network at one time. ² Include all results where the Number of facts in one test data is 5,20,100

The values highlighted in bold in the tables indicate the best performance among all compared methods for each respective metric

6.3 Comparison of correct answer rate

We analyzed whether there was a difference in the correct answer rates between the proposed method and each baseline. The proposed method and baseline correct answer rates listed in Tables 2 and 4 were used in the analysis. The correct answer rates were set to 0.00 when the models were not executable.

Table 5 presents the results of the Wilcoxon signed-rank sum test on the correct answer rates for the proposed method and each baseline. The Wilcoxon signed-rank sum test was used to determine whether there were differences between the distributions of the two populations in the corresponding samples. The significance level was set at $p < 0.05$.

When the number of facts in one test data was 20 and 100, and when it included all of 5, 20, and 100, there were significant differences between the proposed method and

the baselines. When the number of facts in one test data was five, there were significant differences between the proposed method and DNC and between the proposed method and Seq-DNC-Seq.

6.4 Runtime comparison

We compared the runtimes of inference for the trained models in the proposed network with those of the baselines. Table 6 presents the runtime results for the test set for the.

unknown words (12 words) in Table 1. The number of seconds required to process 1,000 facts was measured as a metric. An Intel Core i7-10750H CPU and Nvidia GeForce RTX2070 GPU were used in experiments. The time command of Linux was used to measure the time. The runtime is the sum of the user and system times. The shorter the runtime, the more efficient the network.

We measured the runtime for each matching and substitution network in the proposed network. In the proposed network, only facts that the matching network determines

can be matched are processed by the substitution network. Furthermore, we measured the runtimes of baseline DNC, Seq-DNC-Seq, and Seq2Seq with attention. The Prolog system was excluded from the baseline because it could not handle unknown words.

Table 7 presents the estimated number of seconds required to unify a term with reference to a knowledge base of size 1,000,000 in the proposed network and the baselines. Suppose there are five facts in the knowledge base that match the term. The time required to unify the term and a fact was calculated by referring to Table 6.

6.5 Discussion

In the Kinsources experiments, the correct answer rate of the proposed network was higher than that of the baseline when the number of facts in one test data was 20 or 100. The proposed network had the highest correct answer rate for known and unknown words (4 words), even when the number of facts was five. Even for unknown words (12 words), the correct answer rate was comparable to that of the Seq2Seq with attention, which

Table 6 Runtimes of inference for the trained models in the proposed network and in the baselines. This table lists the runtime results for the test set for the unknown words (12 words) in Table 1

Number of facts in one test data ¹	5	20	100
DNC	65.52	60.85	55.71
Seq-DNC-Seq	72.24	65.02	64.22
Seq2Seq with Attention	144.08	127.30	Not executable
Matching Networks Only (Proposed network)	66.20	60.47	56.70
Substitution Networks Only (Proposed network)	230.44	226.99	227.99

¹ α In Algorithm 1, the number of facts input into the network at one time

The values highlighted in bold in the tables indicate the best performance among all compared methods for each respective metric

Table 7 Estimated number of seconds required to unify a term by reference to a knowledge base of size 1,000,000 in the proposed network and in the baselines. Suppose there are five facts in the knowledge base that match the term. The time required to unify the term and a fact was calculated by referring to Table 6

Number of facts in one test data ¹	5	20	100
DNC	13,104	3,043	557
Seq-DNC-Seq	14,448	3,251	642
Seq2Seq with Attention	28,816	6,365	Not executable
Proposed network	13,241	3,025	568

¹ $\propto$ In Algorithm 1, the number of facts input into the network at one time

The values highlighted in bold in the tables indicate the best performance among all compared methods for each respective metric

had the highest correct answer rate. However, the correct answer rate of the proposed network was approximately 0.5 for unknown words (4 words) and 0.7 for unknown words (12 words) and a certain number of errors were included. Our analysis showed that most of the errors were due to the proposed network's failure to correctly predict the unknown word "man" and "woman." In the word-distributed representation used, among the known words, no word was similar to the "man" vector in cosine similarity. Therefore, the network output was grammatically incorrect. However, the "woman" vector is similar to the "mother" vector and the "father" vector than to the "female" vector in cosine similarity. Therefore, "woman" is substituted for "mother" or "father" instead of "female" in the network. These errors occurred not only in the proposed network but also in the DNC, Seq-DNC-Seq, Seq2Seq with attention, and Seq2Seq + Seq2Seq. We found that there is a limitation with the possibility of erroneous predictions when inputting unknown words that have similarity to multiple predicates or atoms in the knowledge base or have no similarity in the knowledge base. To overcome this limitation, the word-distributed representation must be improved.

In the IMDb experiments, the correct answer rate of the proposed network was the highest compared to the baseline when the number of facts in one test data was 20 and 100. When the number of facts was five, the proposed network had a correct answer rate comparable to Seq2Seq with attention and Seq2Seq + Seq2Seq, which had the best correct answer rate. In the IMDb experiment, unknown words that were similar to several predicates in the knowledge base or that had no similarity in the knowledge base were not included in the test set. Therefore, the proposed network had a correct answer rate of 0.9 or higher even for the test set for unknown words.

In both the Kinsources and IMDb experiments, DNC and Seq-DNC-Seq were suitable for matching because of their external memory. However, it is thought that the correct answer rate was low because the substitution could not be efficiently processed. Seq2Seq with attention cannot be processed when the number of facts included in a single input increases. This is because the storage capacity of Seq2Seq with attention depends on the number of neurons; therefore, it is unsuitable for handling large amounts of data. In Seq2Seq + Seq2Seq, matching and substitution were performed using Seq2Seq with attention. The experimental results of matching with Seq2Seq + Seq2Seq also showed that as the number of facts in one input increased, the matching was poor. This indicates that Seq2Seq with attention is unsuitable for matching. In these experiments, we implemented Seq2Seq with attention using Tensorflow 1.15. However, in our computer environment (Intel Core i7-10750H, Nvidia GeForce RTX2070), TensorFlow incurred runtime errors when the number of facts included in one input was 100. This may be because with Seq2Seq with attention, the longer the input/output string, the more rapidly the memory consumption increases compared with the DNC. Training time was also longer with Seq2Seq with attention than with the DNC.

Furthermore, for the DNC, Seq-DNC-Seq, Seq2Seq with attention, and Seq2Seq + Seq2Seq the correct answer rate decreased as the number of facts included in one input increased. In particular, the DNC and Seq-DNC-Seq showed many errors when substitution did not occur. This indicates that DNC and Seq-DNC-Seq were

unsuitable for substitution. However, the correct answer rate of our proposed network did not decrease even when the number of facts included in one input increased. By combining the advantages of DNC and Seq2Seq with attention, the proposed network can achieve a higher correct answer rate than the baselines. This is supported by the results of the analysis of the differences in correct answer rates between the proposed method and each baseline.

We compared the proposed network with the Prolog system, in both the Kinsources and IMDb experiments. In the Prolog system, the correct answer rate for the known words was 1.0000, whereas it was 0.0000 for the unknown words. However, the proposed network can answer a test set of unknown words, while the correct answer rate for known words remains high. To handle unknown words in a conventional Prolog system, they must be individually registered in a knowledge base. However, the process is labor-intensive. In the proposed network, unknown words can be handled without labor by using a word-distributed representation. Furthermore, the proposed network can handle cases where the predicates and atoms are unknown. This is because, unlike [8], the proposed network is not given a manually created template in advance when training the model.

In the comparison of prediction runtimes, the runtime per fact decreased as the number of facts input at one time increased in the DNC, Seq-DNC-Seq, Seq2Seq with attention, and the matching network for the proposed network. This can be attributed to a reduction in the network processing overhead. Tables 2 and 4 indicate that Seq2Seq with attention had the second highest correct answer rate after the proposed network. Therefore, we compared the runtime of the proposed method and Seq2Seq with attention. As described in Sect. 5.1, the proposed method performs substitution using the substitution network only for facts that the matching network determines can be matched. When the number of facts in one test data was five, the runtime of the proposed network was shorter than Seq2Seq with attention when the percentage of facts in the knowledge base that matched the input term was 33.80% or less. When the number of facts in one test data was 20, the runtime of the proposed network was shorter than Seq2Seq with attention when the percentage of facts in the knowledge base that could be matched with the input term was 29.44% or less. In both cases, the lower the percentage of facts in the knowledge base that can be matched with the input term, the shorter the runtime of the proposed network. Generally, the number of facts contained in a knowledge base matching a term is often sufficiently small compared to the size of the knowledge base.

The predicted runtime of the proposed method indicates that it can handle one million records in 568 s even if they are singulated brute-force on a single computer. Even big data with large records can be handled by the distributed processing of the proposed method in the cloud. In addition, the correct answer rate of the proposed method for unknown words under these conditions is greater than 0.9965. This enables symbolic processing tasks such as deductive and inductive reasoning using big data, including ambiguous data.

The unification in the proposed method is also expected to contribute to research on LLMs, which have rapidly evolved in recent years. One study on LLMs involved a technique called Retrieval-Augmented Generation (RAG) [55], which incorporates an external knowledge base to generate answers. In RAG, an external knowledge base is vectorized, knowledge similar to the input is retrieved, and responses are generated based on the results. Recently, research has been conducted on GraphRAG [56], which represents an external knowledge base as a knowledge graph to perform sophisticated searches with RAGs. Further advanced reasoning using RAGs in the future will require the use of a large symbolic knowledge base. As the proposed method of unification is highly efficient and robust, it is expected to contribute to inferences with a large symbolic knowledge base containing diverse vocabularies. However, because LLMs can handle a wide range of vocabularies, it is necessary to further improve the efficiency of matching using the proposed method for practical use.

7 Conclusion and future work

This study proposed a network that efficiently performs unification using deep learning. The proposed network combines a memory network and a recurrent neural network. The DNC was used for matching, which required the processing of a large-scale knowledge base, and Seq2Seq with attention was used for substitution, which requires processing such as program synthesis.

The novelty of the proposed network lies in its highly efficient matching and robust substitution. The proposed network enables highly efficient matching by grouping multiple terms and processing them in a DNC. Furthermore, the proposed network can handle unknown words even during substitution because it uses Seq2Seq with attention to perform the substitution. The proposed network is the first to perform all unification processes using deep learning. Using this network, efficient matching can be realized from the knowledge base as well as the substitution of unknown words, which is not possible using conventional methods.

The experimental results show that the proposed network can achieve more efficient unification of ambiguous data than the baseline. However, the proposed method is limited in that it may erroneously predict unknown words, based on the performance of the word-distributed representation. One solution to this limitation is to improve the performance of the pretrained word-distribution representations. However, it is difficult to obtain correct answers using pretrained word-distribution representations. Therefore, a possible solution is to hybridize the proposed network with symbolic AI to enable error correction. As the proposed network deals with symbolic processing, it can be used in combination with the Prolog system. The known errors in the proposed network can be handled through error correction using the Prolog system. If the performance of the word-distribution representation is high, the required error correction can be reduced; we believe that this is a practical possibility.

This study combined symbolic processing with a strong inference ability and deep learning, which can handle large amounts of ambiguous data. This contributes to the realization of complex inferences from large amounts of ambiguous data, which are difficult to achieve in conventional research. Furthermore, the use of unification,

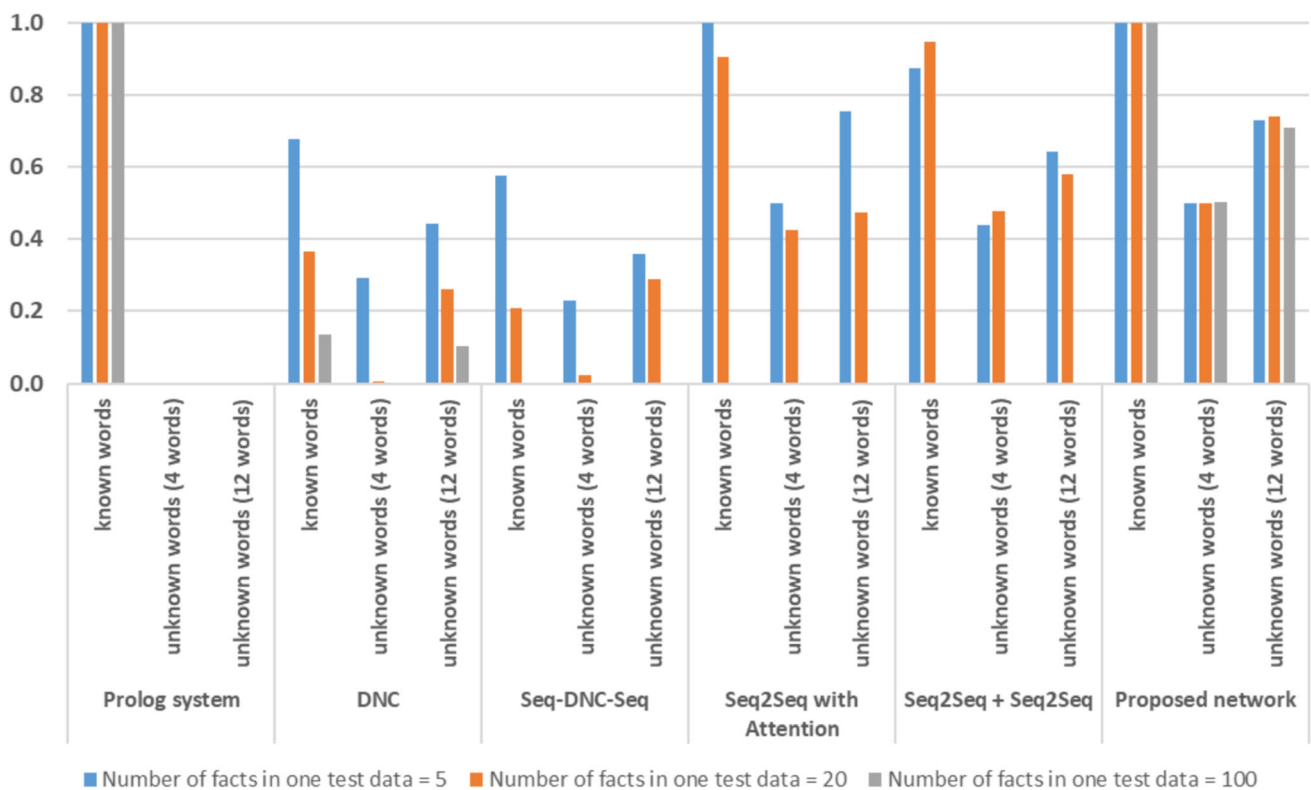


Fig. 9 Correct answer rates for unification using the dataset in Table 1. The figure shows a comparison graph using Table 2

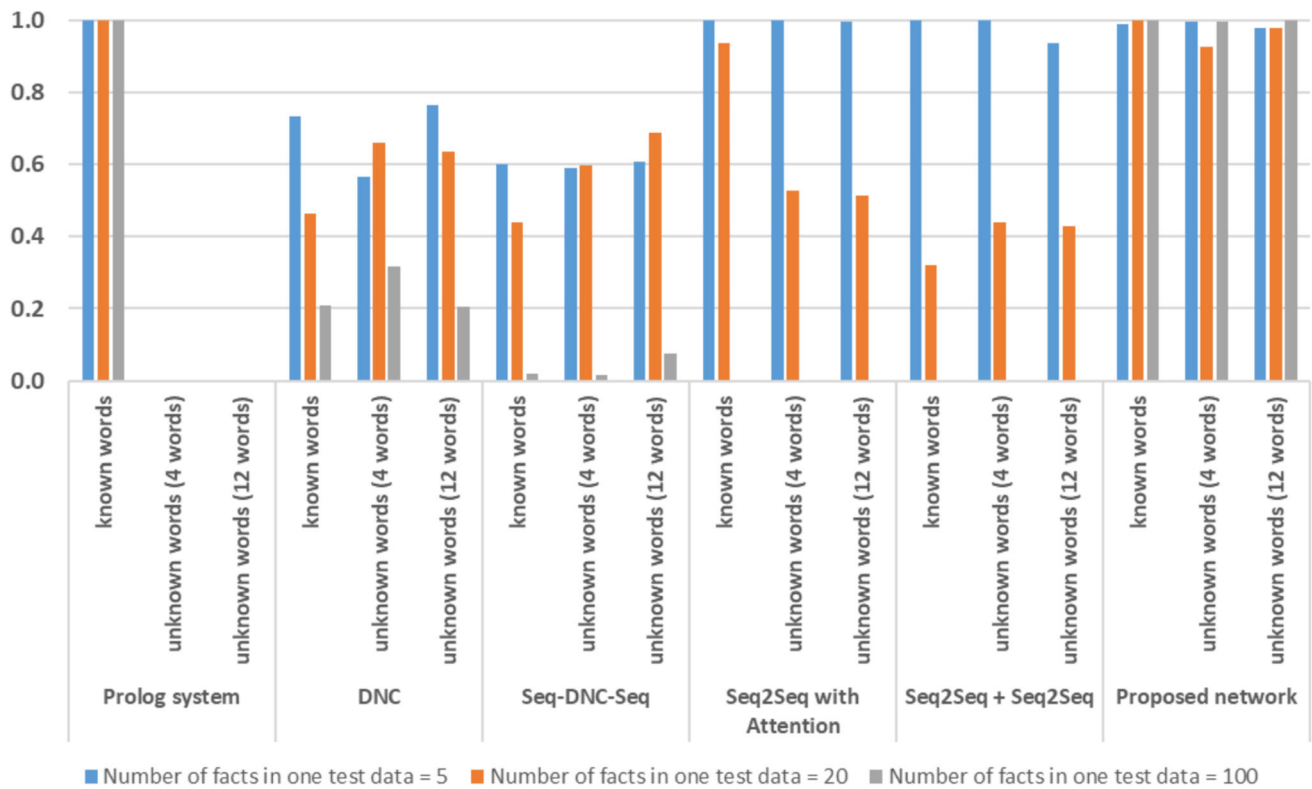


Fig. 10 Correct answer rates for unification using the dataset in Table 3. The figure shows a comparison graph using Table 4

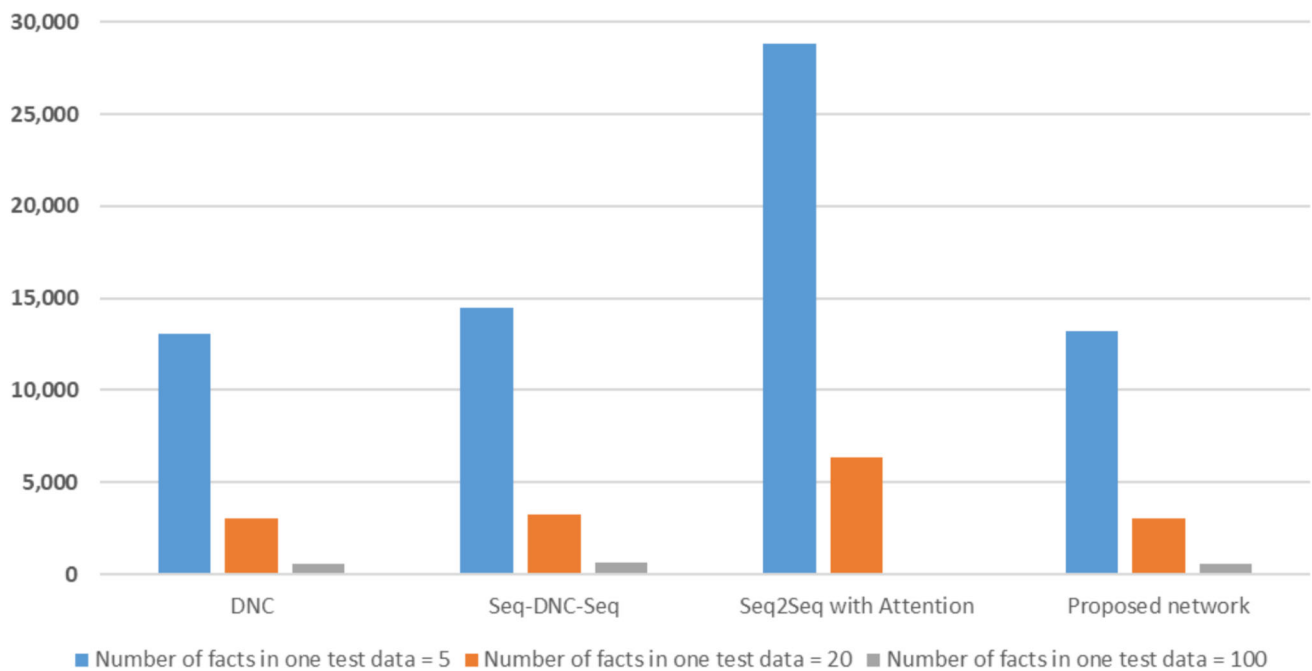


Fig. 11 Estimated number of seconds required to unify a term by reference to a knowledge base of size 1,000,000 in the proposed network and in the baselines. The figure shows a comparison graph using Table 7

which handles large amounts of ambiguous data, enables inference systems with human-interactive interfaces. This allows us to obtain the inference results without knowing the representations of the knowledge base.

Future work will include improving the unification performance by improving the word-distributed representation and unification using large-scale web data. When inference is performed through unification with neural networks, it is difficult to always obtain correct answers, unlike in Prolog systems. Improving the performance of neural networks can address this problem to some extent; however, it still affects the soundness and completeness. This is also true for the study of Neuro-Symbolic AI in general, and we consider this as future work in this field.

Appendix

See Figs. 9, 10, 11.

Author contributions HH designed the study and performed the experiments. HH wrote the manuscript with support from MH. MH supervised this study. All authors discussed the results and contributed to the final manuscript.

Funding Not applicable.

Data availability This study uses publicly available datasets.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

Ethical approval There is no any ethical conflict.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Rivera-Lopez R, Canul-Reich J, Mezura-Montes E, Cruz-Chávez MA (2022) Induction of decision trees as classification models through metaheuristics. *Swarm Evol Comput* 69:101006
2. Law M, Russo A, Bertino E, Broda K, Lobo J (2020) FastLAS: scalable inductive logic programming incorporating domain-specific optimisation criteria. *Proceed AAAI Conf Artif Intell* 34:2877–2885
3. Komo C, Beierle C (2022) Nonmonotonic reasoning from conditional knowledge bases with system W. *Ann Math Artif Intell* 90:107–144
4. Ren Y, Pan JZ (2011) Optimising ontology stream reasoning with truth maintenance system. In: *Proceedings of the 20th ACM international conference on Information and knowledge management*. Association for Computing Machinery, New York pp 831–836
5. Ray O, Kakas A (2006) ProLogICA: a practical system for Abductive Logic Programming. In: *Proceedings of the 11th International Workshop on Non-monotonic Reasoning*. pp 304–312
6. Hinton GE (1990) Preface to the special issue on connectionist symbol processing. *Artif Intell* 46:1–4
7. Touretzky DS (1990) BoltzCONS: dynamic symbol structures in a connectionist network. *Artif Intell* 46:5–46

8. Rocktäschel T, Riedel S (2017) End-to-end differentiable proving. In: Guyon I, Von Luxburg U, Bengio S, Wallach H, Fergus R, Vishwanathan S, Garnett R (eds) *Advances in neural information processing systems*. Curran Associates
9. Honda H, Hagiwara M (2019) Question answering systems with deep learning-based symbolic processing. *IEEE Access* 7:152368–152378
10. Sukhbaatar S, Weston J, Fergus R (2015) End-to-end memory networks. In: Cortes C, Lawrence N, Lee D, Sugiyama M, Garnett R (eds) *Advances in neural information processing systems*. Curran Associates
11. Elman JL (1990) Finding structure in time. *Cogn Sci* 14:179–211
12. Jain P, Hitzler P, Sheth AP, Verma K, Yeh PZ (2010) Ontology Alignment for Linked Open Data. In: Patel-Schneider PF, Pan Y, Hitzler P, Mika P, Zhang L, Pan JZ, Horrocks I, Glimm B (eds) *The semantic web—ISWC 2010*. Springer, Berlin, Heidelberg, pp 402–417
13. Mohammadi M, Rezaei J (2020) Evaluating and comparing ontology alignment systems: an MCDM approach. *J Web Semant* 64:100592
14. Mai G, Huang W, Cai L, Zhu R, Lao N (2022) Narrative Cartography with knowledge graphs. *J Geovis Spat Anal* 6:4
15. Bratko I (2001) *Prolog programming for artificial intelligence*. Pearson education
16. Shavlik JW, Towell GG (1991) An approach to combining explanation-based and neural learning algorithms. *Applications of learning and planning methods*. World Scientific, Singapore, pp 71–98
17. Towell GG, Shavlik JW (1994) Knowledge-based artificial neural networks. *Artif intell* 70:119–165
18. Avila Garcez AS, Zaverucha G (1999) The connectionist inductive learning and logic programming system. *Appl Intell* 11:59–77
19. Shastri L (1999) Advances in SHRUTI—a neurally motivated model of relational knowledge representation and rapid inference using temporal synchrony. *Appl Intell* 11:79–108
20. Ding L (1995) Neural prolog—the concepts, construction and mechanism. 1995 IEEE International conference on systems man and cybernetics intelligent systems for the 21st century. IEEE, Vancouver, pp 3603–3608
21. França MV, Zaverucha G, d’Avila Garcez AS (2014) Fast relational learning using bottom clause propositionalization with artificial neural networks. *Mach Learn* 94:81–104
22. Komendantskaya E (2011) Unification neural networks: unification by error-correction learning. *Logic J IGPL* 19:821–847
23. Hiilddobler S (1990) A structured connectionist unification algorithm. *Proc Natl Conf Am Assoc Artif Intell* 90:587–593
24. Sourek G, Aschenbrenner V, Zelezny F, Schockaert S, Kuzelka O (2018) Lifted relational neural networks: efficient learning of latent relational structures. *J Artif Intell Res* 62:69–100
25. Cohen W, Yang F, Mazaitis KR (2020) TensorLog: a probabilistic database implemented using deep-learning infrastructure. *J Artif Intell Res* 67:285–325
26. Serafini L, Garcez A d’Avila (2016) Logic Tensor Networks: Deep Learning and Logical Reasoning from Data and Knowledge. *Proceedings of the 11th International Workshop on Neural-Symbolic Learning and Reasoning* 1768:
27. Honda H, Hagiwara M (2021) Analogical reasoning with deep learning-based symbolic processing. *IEEE Access* 9:121859–121870
28. Honda H, Hagiwara M (2022) Deep-learning-based fuzzy symbolic processing with agents capable of knowledge communication. *ICAART* 3:172–179
29. Manhaeve R, Dumancic S, Kimmig A, Demeester T, De Raedt L (2018) Deepproblog: Neural probabilistic logic programming. *advances in neural information processing systems* 31:
30. Winters T, Marra G, Manhaeve R, De Raedt L (2022) Deepstochlog: neural stochastic logic programming. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. pp 10090–10100
31. Yang Z, Ishay A, Lee J (2020) Neurasp: embracing neural networks into answer set programming. In: *29th International Joint Conference on Artificial Intelligence (IJCAI 2020)*
32. Hu Y, Yu Y (2022) Enhancing neural mathematical reasoning by abductive combination with symbolic library. *arXiv preprint [arXiv:2203.14487](https://arxiv.org/abs/2203.14487)*
33. Maene J, De Raedt L (2023) Soft-unification in deep probabilistic logic. *Adv Neural Inf Process Syst* 36:60804–60820
34. Picco G, Hoang TL, Sbodio ML, Lopez V (2021) Neural unification for logic reasoning over natural language. In: *Findings of the association for computational linguistics: EMNLP 2021*. Association for computational linguistics, Punta Cana, Dominican Republic, pp 3939–3950
35. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I (2017) Attention is all you need. In: Guyon I, Von Luxburg U, Bengio S, Wallach H, Fergus R, Vishwanathan S, Garnett R (eds) *Advances in neural information processing systems*. Curran Associates Inc
36. De Raedt L, Kimmig A (2015) Probabilistic (logic) programming concepts. *Mach Learn* 100:5–47
37. Hammond K, Leake D (2023) Large language models need symbolic AI. In: *Proceedings of the 17th International Workshop on Neural-Symbolic Learning and Reasoning*, La Certosa di Pontignano, Siena, Italy. pp 204–209
38. Yang S, Li X, Cui L, Bing L, Lam W (2023) Neuro-Symbolic Integration Brings Causal and Reliable Reasoning Proofs. *arXiv preprint [arXiv:2311.09802](https://arxiv.org/abs/2311.09802)*
39. Xiong S, Payani A, Kompella R, Fekri F (2024) Large Language Models Can Learn Temporal Reasoning. *arXiv preprint [arXiv:2401.06853](https://arxiv.org/abs/2401.06853)*

40. Kumar A, Irsoy O, Ondruska P, Iyyer M, Bradbury J, Gulrajani I, Zhong V, Paulus R, Socher R (2016) Ask me anything: dynamic memory networks for natural language processing. *International conference on machine learning*. PMLR, New York, pp 1378–1387
41. Graves A, Wayne G, Danihelka I (2014) Neural turing machines. *arXiv preprint [arXiv:1410.5401](https://arxiv.org/abs/1410.5401)*
42. Graves A, Wayne G, Reynolds M, Harley T, Danihelka I, Grabska-Barwińska A, Colmenarejo SG, Grefenstette E, Ramalho T, Agapiou J (2016) Hybrid computing using a neural network with dynamic external memory. *Nature* 538:471–476
43. Kang D, Lee M (2019) Seq-DNC-seq: context aware dialog generation system through external memory. 2019 International joint conference on neural networks (IJCNN). IEEE, Budapest, Hungary, pp 1–8
44. Gray F (1953) Pulse code communication. United States Patent Number 2632058
45. Mikolov T, Chen K, Corrado G, Dean J (2013) Efficient estimation of word representations in vector space. *arXiv preprint [arXiv:1301.3781](https://arxiv.org/abs/1301.3781)*
46. Mikolov T, Sutskever I, Chen K, Corrado GS, Dean J (2013) Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems* 26:
47. Mikolov T, Yih W, Zweig G (2013) Linguistic regularities in continuous space word representations. In: *Proceedings of the 2013 conference of the North American chapter of the association for computational linguistics: Human language technologies*. pp 746–751
48. Hochreiter S, Schmidhuber J (1997) Long short-term memory. *Neural Comput* 9:1735–1780
49. Tieleman T, Hinton G (2012) Lecture 6.5-rmsprop: divide the gradient by a running average of its recent magnitude. COURSE: Neural networks mach lear 4:26–31
50. Sutskever I, Vinyals O, Le QV (2014) Sequence to sequence learning with neural networks. *Advances in neural information processing systems* 27:
51. Bahdanau D, Cho K, Bengio Y (2014) Neural machine translation by jointly learning to align and translate. *arXiv preprint [arXiv:1409.0473](https://arxiv.org/abs/1409.0473)*
52. Kingma DP, Ba J (2014) Adam: A method for stochastic optimization. *arXiv preprint [arXiv:1412.6980](https://arxiv.org/abs/1412.6980)*
53. Kinsources. <https://www.kinsources.net/>. Accessed 5 Jan 2023
54. Frasconi P, Costa F, De Raedt L, De Grave K (2014) klog: a language for logical and relational learning with kernels. *Artif Intell* 217:117–143
55. Lewis P, Perez E, Piktus A et al (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. *Adv Neural Inf Process Syst* 33:9459–9474
56. Edge D, Trinh H, Cheng N, Bradley J, Chao A, Mody A, Truitt S, Larson J (2024) From local to global: a graph RAG approach to query-focused summarization. *arXiv preprint [arXiv:2404.16130](https://arxiv.org/abs/2404.16130)*

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Hiroshi Honda¹  · Masafumi Hagiwara¹

✉ Hiroshi Honda

honda@keio.jp

Masafumi Hagiwara

hagiwara@keio.jp

¹ Faculty of Science and Technology, Keio University, Yokohama, Japan